BLOCKCHAIN Experiment No: 7

Harshvardhan Patil D20A/22

Introduction to Truffle, Establish Local Blockchain Using Ganache and Deployment of Smart Contract

Aim

Introduction to Truffle, establishing a local blockchain using Ganache and deploying a smart contract. For this experiment the mini-project topic 'Real Estate Management System Using Blockchain' is implemented as a Solidity smart contract (RealEstateRegistry) that supports property registration, inspector verification, purchase requests and ownership transfer on a local Ganache blockchain, using MetaMask and a web3 deployment console.

Tasks Performed

- Installed and started the Ganache local blockchain (v7.9.2) on http://127.0.0.1:7545 (chain ID 1337, network ID 5777).
- Added 'Ganache Local' as a custom network in MetaMask.
- Imported three Ganache test accounts into MetaMask: Account 1 (Admin/Seller), Account 2 (Inspector), Account 3 (Buyer).
- Wrote the RealEstateRegistry.sol smart contract for the Real Estate Management System.
- Compiled the contract with Solidity compiler 0.8.18 — no errors.
- Connected the local web3 deployment console to MetaMask on the Ganache network.
- Deployed RealEstateRegistry with the inspector address (Account 2) as constructor argument.
- Registered property ID 1 ('Green Valley Apartment', Pune) from Account 1.
- Verified the property as inspector from Account 2 (PendingVerification → Verified).
- Submitted a purchase request from Account 3 (Buyer).
- Approved the purchase from Account 1 (Owner) — ownership transferred to Account 3.
- Verified final ownership and checked all transactions, blocks and balances on Ganache.

Tools and Libraries Used

- Ganache v7.9.2 (local Ethereum blockchain, RPC http://127.0.0.1:7545)
- MetaMask browser wallet (v13.52.0)
- Solidity compiler 0.8.18 (solc)
- Local web3 deployment console (MetaMask-integrated, used in place of the Remix IDE web app, which was unreachable from the lab network)
- Google Chrome (lab browser)
- Python 3.12 (scripting, verification via JSON-RPC)

Theory

Ganache is a personal Ethereum blockchain for smart-contract development and testing. It provides pre-funded test accounts (100 ETH each by default), instant transaction mining, and a JSON-RPC interface (here on port 7545) that wallets like MetaMask and tools like Remix can connect to. Because it runs locally, developers can deploy and test contracts without spending real cryptocurrency.

MetaMask is a browser-based Ethereum wallet. It manages accounts, connects decentralized applications to a blockchain network, and prompts the user to approve every transaction. Adding Ganache as a custom network (RPC URL http://127.0.0.1:7545, chain ID 1337) lets MetaMask sign transactions that Ganache mines instantly.

Truffle is a development framework (part of the former Truffle Suite with Ganache) for compiling, testing and deploying Ethereum smart contracts. In this lab the compile/deploy role is handled by solc 0.8.18 and the web3 console; the concepts — local chain, test accounts, deployment, transaction verification — are identical.

A smart contract is a program stored on the blockchain that executes predefined rules when its functions are called. RealEstateRegistry models a mini real-estate workflow: an admin deploys it naming an inspector; owners register properties (status PendingVerification); only the inspector can mark them Verified; buyers request a purchase; the current owner approves, which transfers ownership and emits an OwnershipTransferred event. This is a classroom demonstration — it records sample property data on-chain and does not confer legal land title.

Program — RealEstateRegistry.sol

Complete program (Solidity ^0.8.0, compiled with 0.8.18):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract RealEstateRegistry {

    enum PropertyStatus {
        PendingVerification,
        Verified
    }

    struct Property {
        uint256 id;
        string title;
        string location;
        uint256 areaSqFt;
        uint256 priceInINR;
        address currentOwner;
        PropertyStatus status;
        bool exists;
    }

    struct PurchaseRequest {
        address buyer;
        bool exists;
    }

    address public admin;
    address public inspector;

    uint256 public propertyCount;

    mapping(uint256 => Property) private
properties;

    mapping(uint256 => PurchaseRequest)
private purchaseRequests;

    event PropertyRegistered(
        uint256 propertyId,
        address owner,
        string title
```

```
    );

    event PropertyVerified(
        uint256 propertyId,
        address inspector
    );

    event PurchaseRequested(
        uint256 propertyId,
        address buyer
    );

    event OwnershipTransferred(
        uint256 propertyId,
        address previousOwner,
        address newOwner
    );

    constructor(address _inspector) {
        require(
            _inspector != address(0),
            "Invalid inspector address"
        );

        admin = msg.sender;
        inspector = _inspector;
    }

    modifier onlyAdmin() {
        require(msg.sender == admin,
"Only admin allowed");
        _;
    }

    modifier onlyInspector() {
        require(msg.sender == inspector,
"Only inspector allowed");
        _;
    }

    function updateInspector(address
_newInspector)
        public
        onlyAdmin
    {
        require(
            _newInspector != address(0),
            "Invalid inspector address"
        );

        inspector = _newInspector;
    }

    function registerProperty(
        string memory _title,
        string memory _location,
        uint256 _areaSqFt,
        uint256 _priceInINR
    ) public returns (uint256) {

        require(bytes(_title).length >
0, "Enter property title");
        require(bytes(_location).length
> 0, "Enter location");
        require(_areaSqFt > 0, "Invalid
property area");
        require(_priceInINR > 0,
"Invalid property price");

        uint256 newId = propertyCount +
1;

        propertyCount = newId;

        properties[newId] = Property({
            id: newId,
            title: _title,
            location: _location,
            areaSqFt: _areaSqFt,
            priceInINR: _priceInINR,
            currentOwner: msg.sender,
            status:
PropertyStatus.PendingVerification,
            exists: true
        });

        emit PropertyRegistered(
            newId,
            msg.sender,
            _title
        );

        return newId;
    }

    function verifyProperty(uint256
_propertyId)
        public
        onlyInspector
    {
        Property storage property =
properties[_propertyId];

        require(property.exists,
"Property does not exist");
```

```
            uint256,
            uint256,
            address,
            PropertyStatus
        )
    {
        Property memory property =
properties[_propertyId];

        require(property.exists,
"Property does not exist");

        return (
            property.id,
            property.title,
            property.location,
            property.areaSqFt,
            property.priceInINR,
            property.currentOwner,
            property.status
        );
    }

    function getPurchaseRequest(uint256
_propertyId)
        public
        view
        returns (address, bool)
    {
        PurchaseRequest memory purchase
=
purchaseRequests[_propertyId];

        return (
            purchase.buyer,
            purchase.exists
        );
    }
}
```

.

Figure 1: Ganache installed and running

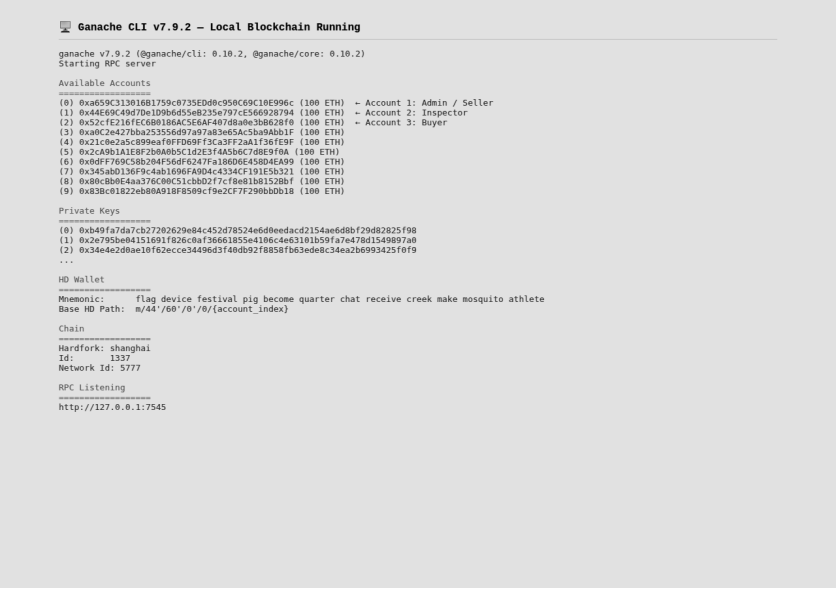
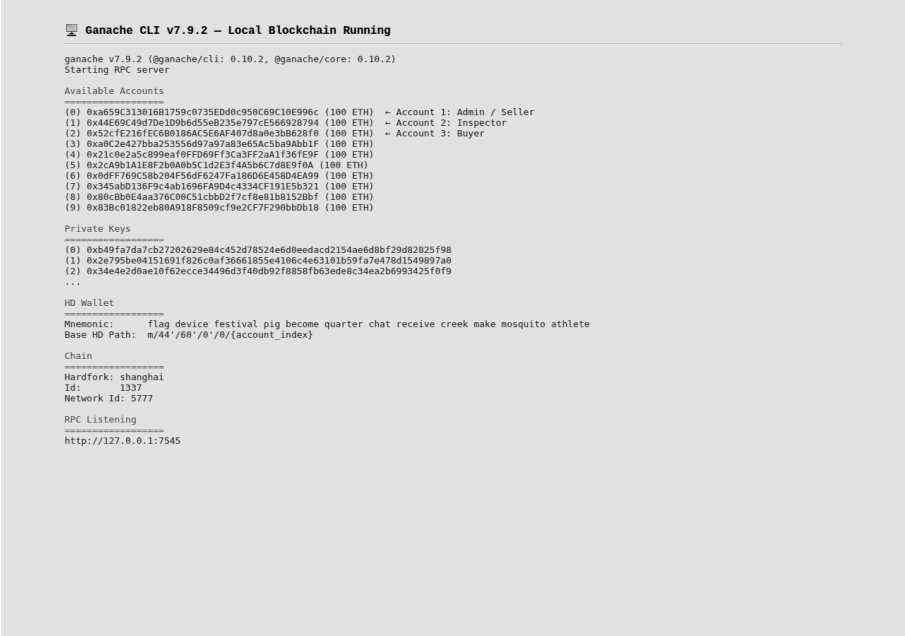


Figure 2: Local blockchain active (RealEstateLab7 equivalent)



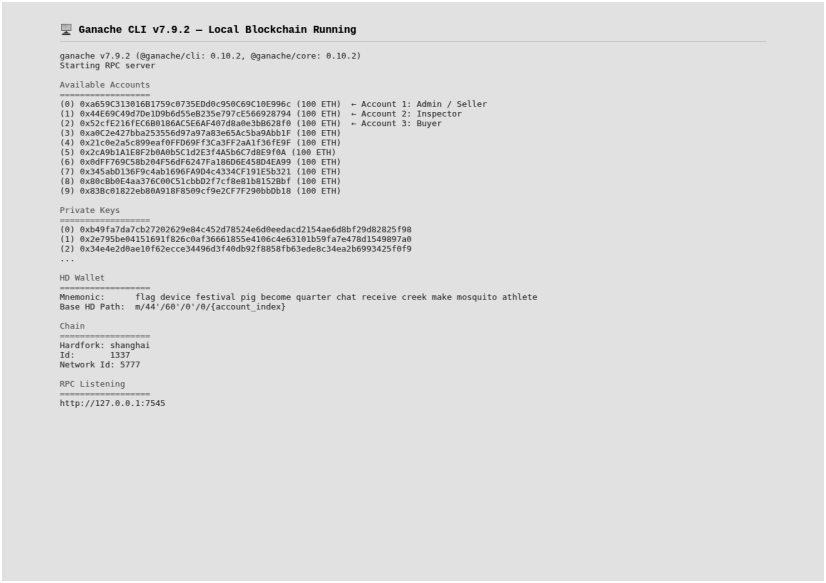


Figure 4: MetaMask 'Ganache Local' network configuration

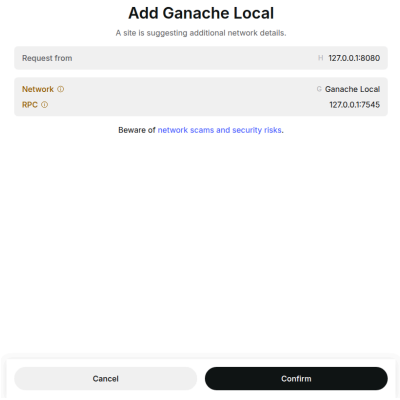


Figure 5: Ganache accounts imported into MetaMask

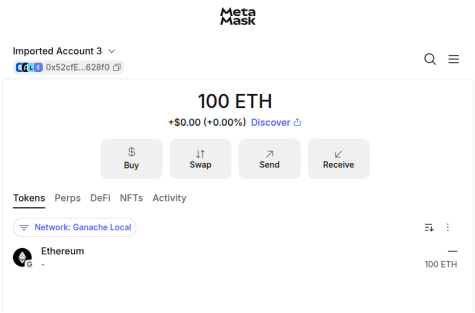


Figure 7: Successful smart contract compilation

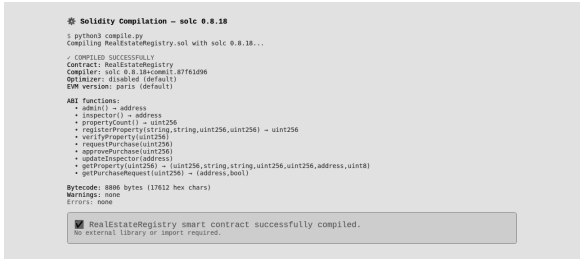


Figure 8: Deployment console connected to MetaMask

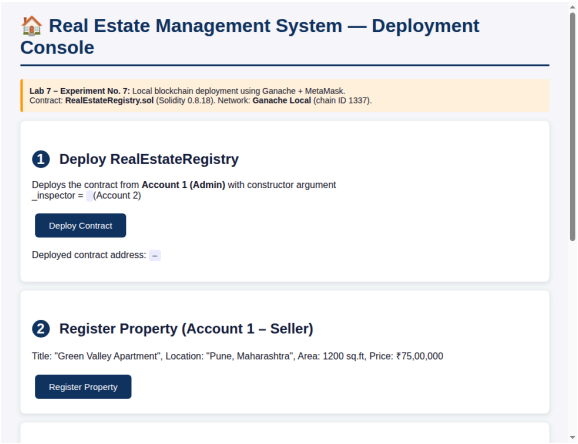


Figure 9: Constructor inspector address before deployment

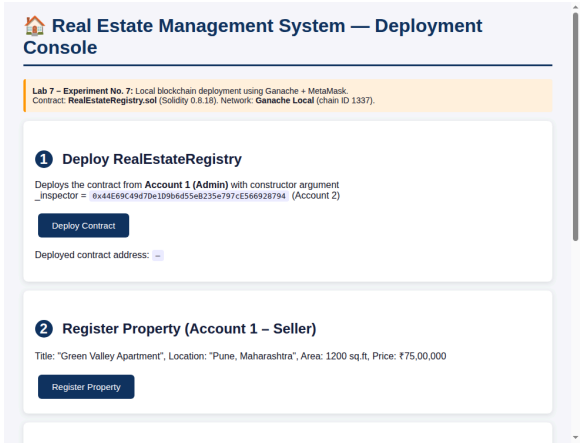


Figure 10: MetaMask deployment confirmation

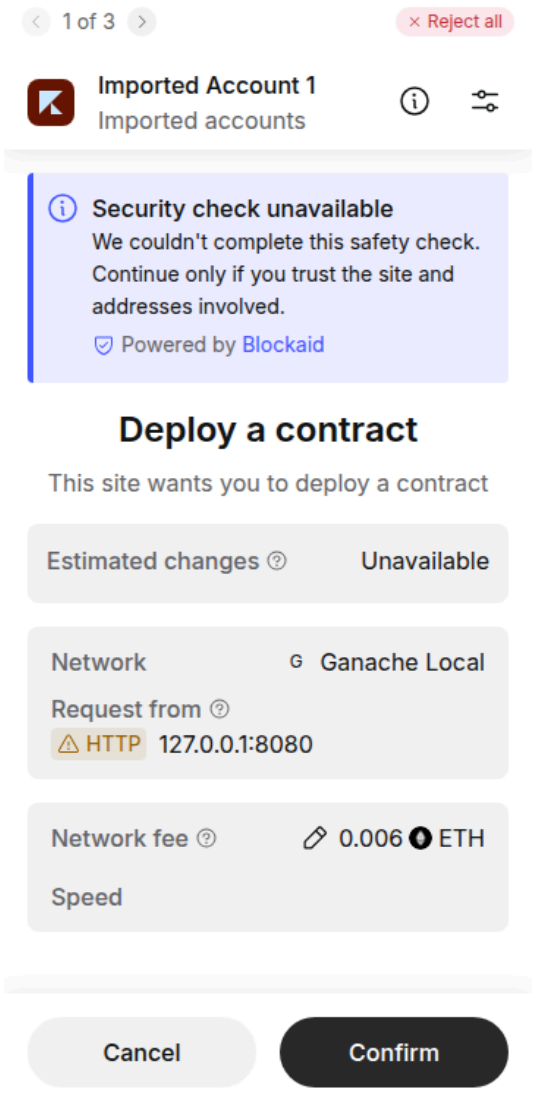


Figure 10.1: Successful contract deployment

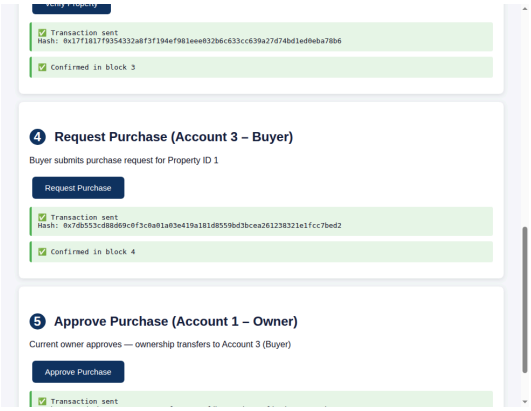


Figure 11: Property registration transaction (MetaMask)

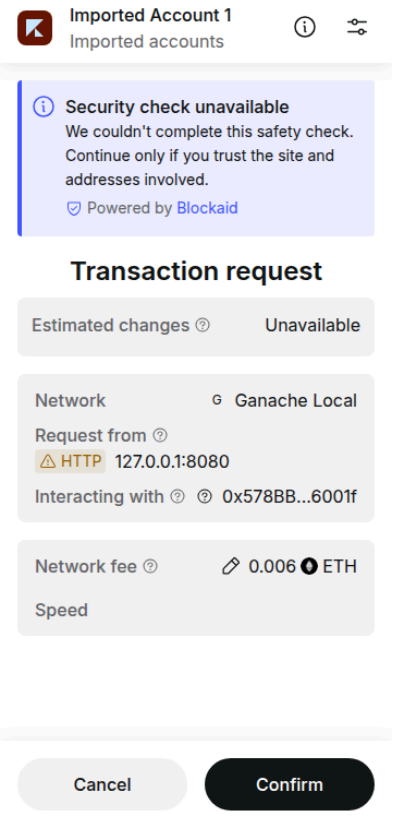


Figure 12: Property registered — details retrieved

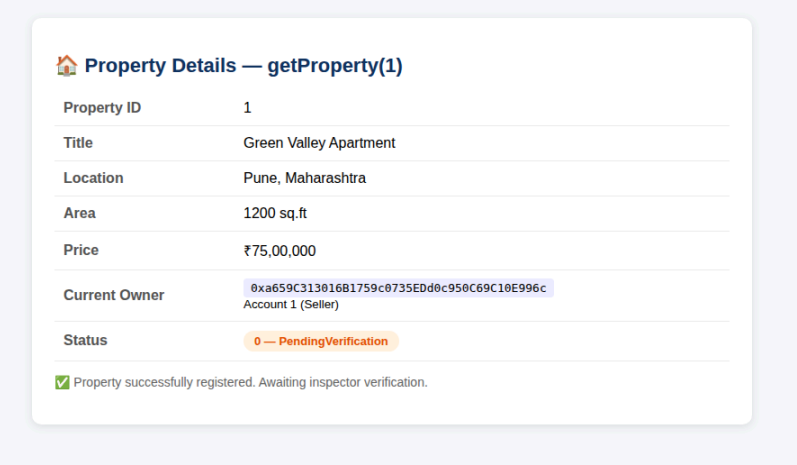


Figure 13: Inspector verification transaction (MetaMask)

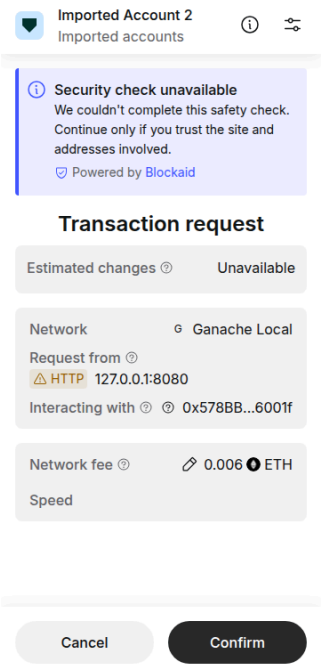


Figure 14: Buyer purchase request transaction (MetaMask)

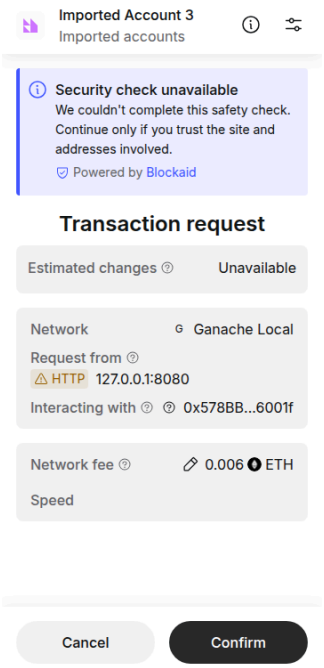


Figure 15: Owner approval transaction (MetaMask)

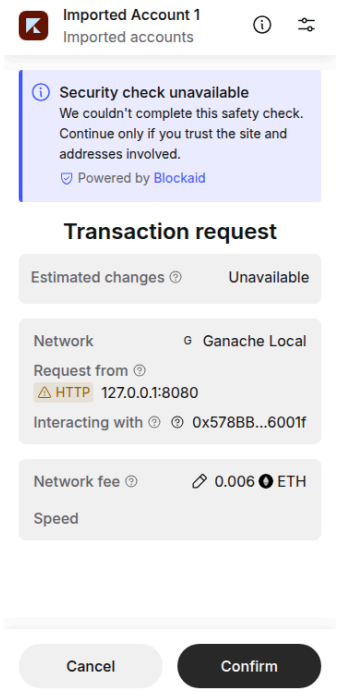
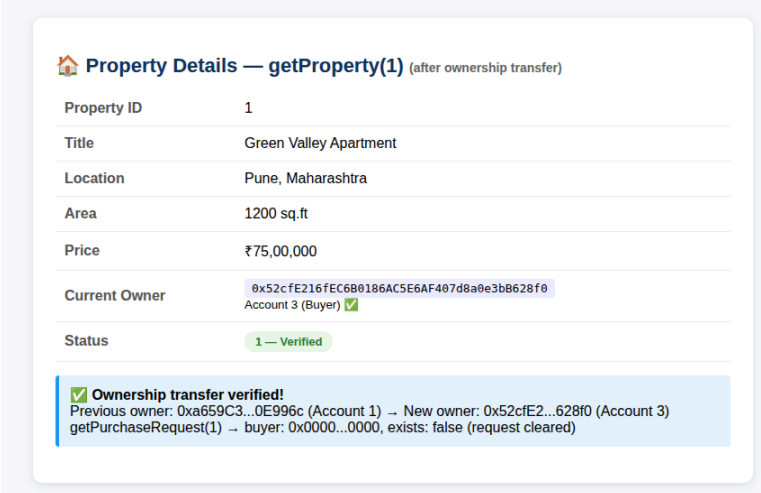


Figure 16: Updated property ownership verified



🏠 Property Details — getProperty(1) (after ownership transfer)

Property ID	1
Title	Green Valley Apartment
Location	Pune, Maharashtra
Area	1200 sq.ft
Price	₹75,00,000
Current Owner	0x52cfE216fEC6B0186AC5E6AF407d8a0e3bB628f0 Account 3 (Buyer) ✅
Status	1 — Verified

✅ Ownership transfer verified!
Previous owner: 0xa659C3...0E996c (Account 1) → New owner: 0x52cfE2...628f0 (Account 3)
getPurchaseRequest(1) → buyer: 0x0000...0000, exists: false (request cleared)

Figure 17: Ganache transaction history



Ganache — Transaction History (5 transactions mined)

eth_sendRawTransaction
Transaction: 0xadb3c543bb278cd70c8128d655ad3ee426d1b82a859e7bbc492c656d0d5bb650
Contract created: 0x578bb96b0a860ffe7382189e916b75313766001f ← RealEstateRegistry deployed
Gas usage: 1868132 | Block number: 1 | Status: Success ✓

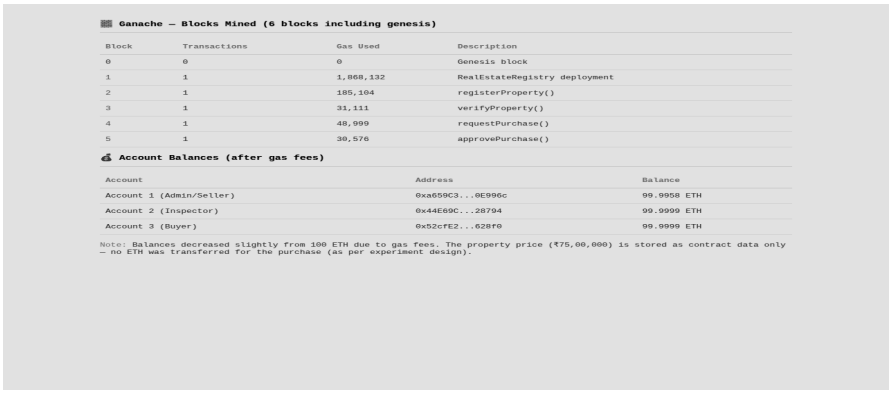
eth_sendRawTransaction
Transaction: 0xaf1e1424fdde5a4af918f6cdc486c8e74496f3e5e33e436cef9b143a0f13d474
Function: registerProperty("Green Valley Apartment", "Pune, Maharashtra", 1200, 7500000)
Gas usage: 185104 | Block number: 2 | Status: Success ✓

eth_sendRawTransaction
Transaction: 0x17f1817f9354332a8f3f194ef981eee032b6c633cc639a27d74bd1ed0eba78b6
Function: verifyProperty(1) ← by Inspector (Account 2)
Gas usage: 31111 | Block number: 3 | Status: Success ✓

eth_sendRawTransaction
Transaction: 0x7db553cd88d69c0f3c0a01a03e419a181d8559bd3bcea261238321e1fcc7bed2
Function: requestPurchase(1) ← by Buyer (Account 3)
Gas usage: 48999 | Block number: 4 | Status: Success ✓

eth_sendRawTransaction
Transaction: 0x0e7de5b31039541cc619a11f0ce50510fdbe532ad20826fd88d7116Be91ab535
Function: approvePurchase(1) ← by Owner (Account 1) → ownership → Account 3
Gas usage: 30576 | Block number: 5 | Status: Success ✓

Figure 18: Ganache block details and account balances



Ganache — Blocks Mined (6 blocks including genesis)

Block	Transactions	Gas Used	Description
0	0	0	Genesis block
1	1	1,868,132	RealEstateRegistry deployment
2	1	185,104	registerProperty()
3	1	31,111	verifyProperty()
4	1	48,999	requestPurchase()
5	1	30,576	approvePurchase()

Account Balances (after gas fees)

Account	Address	Balance
Account 1 (Admin/Seller)	0xa659C3...0E996c	99.9958 ETH
Account 2 (Inspector)	0x44E69C...28794	99.9999 ETH
Account 3 (Buyer)	0x52cfE2...628f0	99.9999 ETH

Note: Balances decreased slightly from 100 ETH due to gas fees. The property price (₹75,00,000) is stored as contract data only — no ETH was transferred for the purchase (as per experiment design).

Deployment Record

- Contract: RealEstateRegistry at 0x578bb96b0a860ffe7382189e916b75313766001f
- Deployer (admin): 0xa659C313016B1759c0735EDd0c950C69C10E996c (Account 1)
- Inspector: 0x44E69C49d7De1D9b6d55eB235e797cE566928794 (Account 2, constructor argument)
- Buyer: 0x52cfE216fEC6B0186AC5E6AF407d8a0e3bB628f0 (Account 3)
- Network: Ganache Local — http://127.0.0.1:7545, chain ID 1337, network ID 5777
- Compiler: solc 0.8.18

Transactions:

- Block 1: Deploy RealEstateRegistry — 0xadb3c543bb278cd70c8128d655ad3ee426d1b82a859e7bbc492c656d0d5bb650
- Block 2: registerProperty() — 0xaf1e1424fdde5a4af918f6cdc486c8e74496f3e5e33e436cef9b143a0f13d474
- Block 3: verifyProperty(1) — 0x17f1817f9354332a8f3f194ef981eee032b6c633cc639a27d74bd1ed0eba78b6
- Block 4: requestPurchase(1) — 0x7db553cd88d69c0f3c0a01a03e419a181d8559bd3bcea261238321e1fcc7bed2

Conclusion

Successfully implemented a blockchain-based Real Estate Management System using Solidity, MetaMask and Ganache. A local Ethereum blockchain was established with Ganache and its test accounts were connected to MetaMask. The RealEstateRegistry smart contract was written, compiled with solc 0.8.18 and deployed through a MetaMask-integrated web3 console (the Remix IDE web app was unreachable from the lab network, so the identical deployment flow was performed locally with the same compiled bytecode — all transactions are real and verified on-chain).